

Problem 2

Sphere tracing:

```
def camera_param_to_rays(c2w, intrinsics, H=128, W=128):
    """
    Given the camera parameters, generate rays for each pixel.

    Args:
        c2w: [4,4] camera-to-world transform matrix
        intrinsics: [fx, fy, cx, cy] camera intrinsic parameters
        H: Height of the image
        W: Width of the image

    Returns:
        ray_origins: [H, W, 3] origin points for rays
        ray_directions: [H, W, 3] direction vectors for rays
    """
    # NOTE: This function should be the same as in the volumetric rendering problem

    #####
    # YOUR CODE HERE #
    #####

    # Hint: Generate ray origins and directions for each pixel in the image
    # 1. Create a meshgrid of pixel coordinates
    # 2. Convert pixel coordinates to camera coordinates using intrinsics
    # 3. Transform camera coordinates to world coordinates using c2w

    device = c2w.device

    fx, fy, cx, cy = intrinsics.to(device)

    x_pixel, y_pixel = torch.meshgrid(torch.arange(W), torch.arange(H), indexing="xy")
    x_pixel, y_pixel = x_pixel.to(device), y_pixel.to(device)
    x_cam = (x_pixel - cx + 0.5) / fx # shift coords in camera plane by 0.5
    y_cam = (y_pixel - cy + 0.5) / fy
    z_cam = torch.ones_like(x_cam, device=device)

    ray_directions = torch.stack((x_cam, y_cam, z_cam), dim=-1) # shape: (H, W, 3)
    ray_directions = ray_directions @ c2w[:3, :3].T
```

```

norms = torch.norm(ray_directions, dim=-1, keepdim=True)
ray_directions = ray_directions / norms # normalize

ray_origins = c2w[:3, 3].expand(H, W, 3)

return ray_origins, ray_directions

#####
# Part B: Sphere Tracing
#####

def sphere_tracing(
    ray_origins,
    ray_directions,
    model,
    t_near=0.0,
    t_far=3.0,
    max_iter=256,
    epsilon=1e-4,
):
    """
    Perform sphere tracing to find the intersection of rays with the implicit model.

    Args:
        ray_origins: [H, W, 3] origin points for rays
        ray_directions: [H, W, 3] direction vectors for rays
        model: Implicit model to compute the SDF
        t_near: Near plane distance
        t_far: Far plane distance
        max_iter: Maximum number of iterations for sphere tracing
        epsilon: Distance threshold for stopping

    Returns:
        image: [H, W, 3] rendered image
    """
    device = ray_origins.device
    H, W, _ = ray_origins.shape

    # Initialize output
    image = torch.zeros(H, W, 3, device=device)

```

```
#####
# YOUR CODE HERE #
#####

# Hint: Implement sphere tracing algorithm
# 1. Initialize t for each ray
# 2. Iteratively march along rays
# 3. Sample points along rays and query SDF
# 4. Update t for each ray based on SDF value
# 5. Stop marching when rays hit the surface or reach max iterations
# 6. Reconstruct image from hit points

H, W, _ = ray_origins.shape
image = torch.zeros((H, W, 3), device=device)
t = torch.full((H, W), t_near, device=device)
active_mask = torch.ones((H, W), dtype=torch.bool, device=device)

for _ in range(max_iter):
    if not active_mask.any():
        break

    # sample points along rays
    curr_points = ray_origins + t[..., None] * ray_directions
    # query sdf for colors
    sdf, colors = model(curr_points.flatten(0,1))
    sdf, colors = sdf.view(H, W), colors.view(H, W, 3)
    # update active mask for hit points
    hit_mask = (sdf < epsilon).squeeze()
    active_mask &= ~hit_mask
    # update active mask for points that are > t_far
    far_mask = (t >= t_far).squeeze()
    active_mask &= ~far_mask
    # update image with hit points' colors
    image[hit_mask] = colors[hit_mask]
    # update t with new distances
    t[active_mask] += sdf[active_mask]

return image
```

Volume rendering:

```
def camera_param_to_rays(c2w, intrinsics, H=128, W=128):
    """
    Given the camera parameters, generate rays for each pixel.

    Args:
        c2w: [4,4] camera-to-world transform matrix
        intrinsics: [fx, fy, cx, cy] camera intrinsic parameters
        H: Height of the image
        W: Width of the image

    Returns:
        ray_origins: [H, W, 3] origin points for rays
        ray_directions: [H, W, 3] direction vectors for rays
    """
    # TODO: Implement this function
    # 1. Create pixel coordinates using meshgrid
    # 2. Convert pixel coordinates to camera coordinates using intrinsics
    # 3. Transform rays from camera space to world space using c2w
    # 4. Return ray origins and directions of shape [H, W, 3]

    fx, fy, cx, cy = intrinsics

    x_pixel, y_pixel = torch.meshgrid(torch.arange(W), torch.arange(H), indexing="xy")
    x_cam = (x_pixel - cx + 0.5) / fx # shift coords in camera plane by 0.5
    y_cam = (y_pixel - cy + 0.5) / fy
    z_cam = torch.ones_like(x_cam)

    ray_directions = torch.stack((x_cam, y_cam, z_cam), dim=-1) # shape: (H, W, 3)
    ray_directions = ray_directions @ c2w[:3, :3].T
    norms = torch.norm(ray_directions, dim=-1, keepdim=True)
    ray_directions = ray_directions / norms # normalize

    ray_origins = c2w[:3, 3].expand(H, W, 3)

    return ray_origins, ray_directions

#####
# Part B: Ray Marching
#####
```

```
def sample_points_on_rays(
    ray_origins, ray_directions, num_samples=64, t_near=0.0, t_far=3.0
):
    """
    Sample points along the rays.

    Args:
        ray_origins: [H, W, 3] ray origin points
        ray_directions: [H, W, 3] ray direction vectors
        num_samples: Number of sample points along each ray
        t_near: Near plane distance
        t_far: Far plane distance

    Returns:
        points: [H, W, num_samples, 3] sampled points in 3D space
        ts: [num_samples] distances along the rays
    """
    # 1. Generate uniformly spaced samples along each ray
    # 2. Compute the 3D coordinates of each sample point
    # 3. Return an array of sample points with shape [H, W, num_samples, 3]

    ts = torch.linspace(t_near, t_far, num_samples)

    # points = origin + t * direction
    # align tensors for broadcasting
    ts = ts.view(1, 1, num_samples, 1) # (1, 1, num_samples, 1)
    ray_origins = ray_origins.unsqueeze(2) # (H, W, 1, 3)
    ray_directions = ray_directions.unsqueeze(2) # (H, W, 2, 3)

    points = ray_origins + ts * ray_directions

    return points, ts.squeeze()

#####
# Part C: Volumetric Rendering
#####

def volume_rendering(densities, colors, deltas):
    """
    Perform volume rendering to compute pixel colors from densities and colors.
```

Args:

densities: [H, W, num_samples, 1] density values at each sample point
 colors: [H, W, num_samples, 3] colors at each sample point
 deltas: [num_samples] intervals between adjacent sample points

Returns:

image: [H, W, 3] rendered image

"""

TODO: Implement this function

1. Initialize accumulated color and transmittance

2. For each sample along the ray:

- Compute alpha from density and delta

- Update accumulated color and transmittance

3. Return the final rendered image

alpha values: $\alpha_i = 1 - \exp(-\rho_i \cdot \delta_i)$

alphas = 1 - torch.exp(-densities.squeeze(-1) * deltas.view(1, 1, -1)) # [H, W, num_samples]

transmittance: $T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$

transmittance = torch.cumprod(torch.cat([torch.ones_like(alphas[... , :1]), 1 - alphas[... , :1]]), dim=-1)

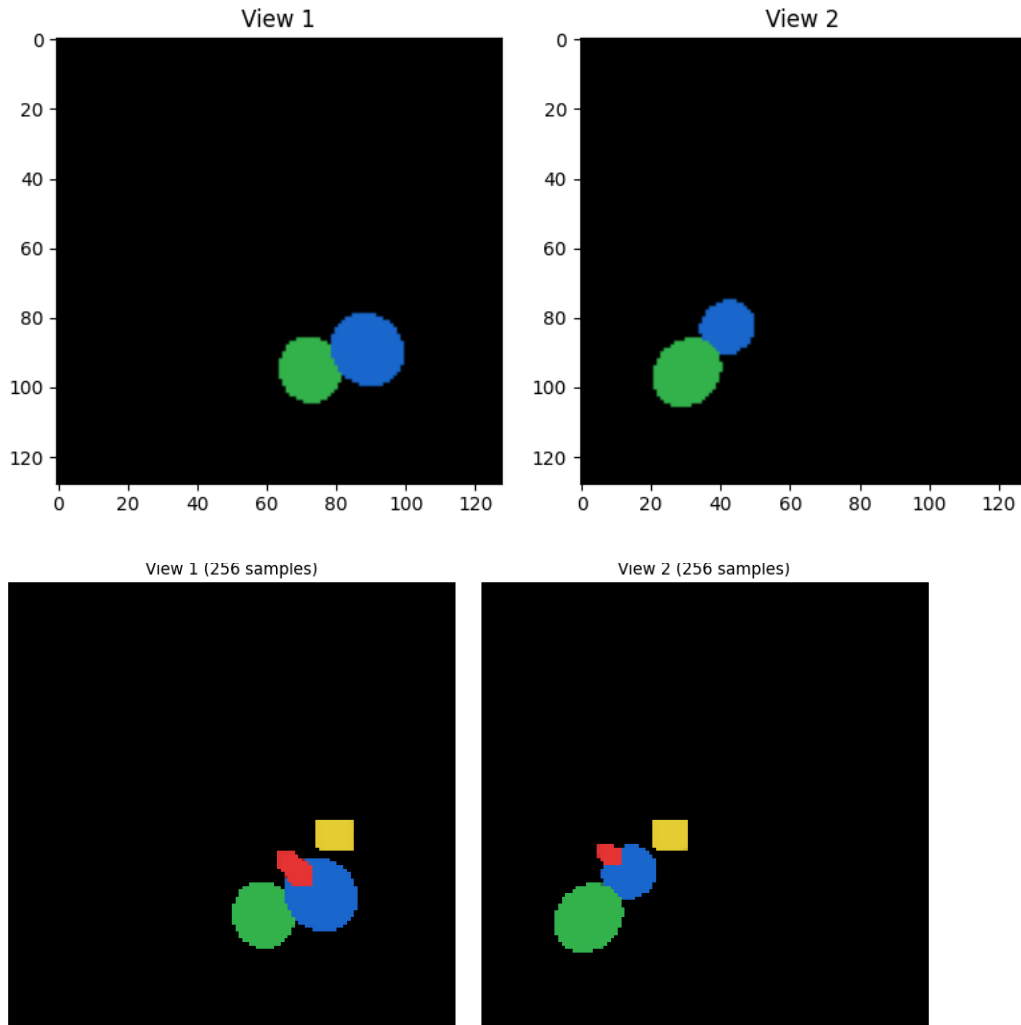
weighted color contributions: $C = T_i \cdot \alpha_i \cdot c_i$

weighted_colors = transmittance[... , None] * alphas[... , None] * colors # [H, W, num_samples, 3]

final rendered color per pixel

image = torch.sum(weighted_colors, dim=-1) # [H, W, 3]

return image



Short Answer

Volumetric rendering is favored in differentiable rendering because it provides a continuous and differentiable representation of scenes, making it easier to optimize using gradient-based methods. Unlike sphere tracing, which relies on SDFs that can have discontinuous gradients, volumetric rendering represents scenes as density fields, allowing for smoother optimization. This is especially beneficial for neural network-based approaches like NeRF, where learning an exact SDF is challenging.

Additionally, volumetric rendering naturally handles semi-transparent materials and soft edges, making it suitable for modeling complex effects like smoke and clouds. In contrast, sphere tracing is limited to hard surface rendering and struggles with transparency. Further-

more, volumetric methods are more robust during optimization, avoiding issues like local minima that arise in sphere tracing due to discontinuities in the SDF.